

ASSIGNMENT

Power BI: Framework Monitor

for Rodrigo

Prepared by Data Simplified (BV)

Date: 30/03/2026

Version: 1.0

Confidential

Table of Contents

Table of Contents.....	2
1. Introduction	3
2. Dataset Context: The DS Framework.....	4
2.1 Pipeline Flow.....	4
2.2 Logging & Monitoring	4
2.3 Why PL_3 Data is One Day Behind	4
3. Data Model.....	5
3.1 Fact Tables	5
3.2 Dimension Tables	5
3.3 Relationships	6
4. Assignment Deliverables	7
4.1 Measures: Clean Foundation.....	7
Critical: Filter to Night Loads Only.....	7
Interesting Measures (or that we think could be interesting)	7
4.2 Source Applications Overview	8
Data Quality sub-page.....	8
4.3 Front Sheet: Per-Customer Pipeline Duration	9
Requirements	9
Visual Design Hint	9
4.4 Long-Running Loads Analysis	10
Requirements	10
Easy Improvement Indicators	10
4.5 Metadata Tables & Settings	11
5. Deliverables Summary.....	12
5.1 Approach	12
6. Resources.....	13
6.1 Power BI Visual Design	13
6.2 DAX Reference	13
6.3 DS Framework Documentation.....	13
Appendix A: DAX Coding Standards	14
A.1 Formatting & Indentation	14
A.2 Naming Conventions.....	14
A.3 Variable Usage.....	14
A.4 Division.....	14
A.5 Comments.....	15
A.6 Measure Organisation	15
A.7 Visual Design Rules.....	15
A.8 Conditional Formatting Colors	15

1. Introduction

Welcome to the Framework Monitor Power BI assignment. This document describes the dataset, the data model, and the expected deliverables. The goal is to develop a monitoring report in Power BI Desktop that provides insight into the nightly ETL performance of our clients' data platforms.

You will be working in the file **Data Simplified - Framework Monitor - Rodrigo.pbix**. This file connects to the DS framework's logging layer (log schema) via Import-mode.

This assignment is designed to help you become comfortable with Power BI report development, DAX measure authoring, and the consultancy mindset: asking the right questions before building.

Consultant mindset

In real projects, requirements are rarely 100% clear. If something is ambiguous, ask clarifying questions. In this assignment, Data Simplified (Bavo) acts as your 'customer'. Use Teams or a direct message to ask questions. This is expected and encouraged.

2. Dataset Context: The DS Framework

Data Simplified builds centralized data platforms for clients on Microsoft Azure. The core of these platforms is the DS Framework, a metadata-driven ETL architecture that runs as a nightly Azure Data Factory (ADF) pipeline flow.

2.1 Pipeline Flow

Every night, the framework executes three main pipeline stages in sequence:

Pipeline	Stage	Purpose
PL_1_Ingestion	Ingestion	Copies source data into the SQL database (Bronze layer). Sources include SQL databases, APIs, SharePoint, SAP, etc.
PL_2_Datawarehouse	Transformation	Runs Preloads, Dimensions, and Facts (Silver/Gold layers). All transformation logic is T-SQL stored procedures.
PL_3_Serve	Delivery	Refreshes Power BI datasets, exports CSV files, or triggers downstream processes.

These three pipelines are orchestrated by PL_0_Main, which obtains a unique ExecutionId at the start and logs everything throughout the process.

2.2 Logging & Monitoring

The framework writes detailed trace data to the log schema:

- **log.ExecutionTrace** records one row per pipeline execution (start time, end time, duration, night load flag).
- **log.SQLStatementTrace** records every SQL statement executed during PL_2, including duration in milliseconds, error messages, and print output.

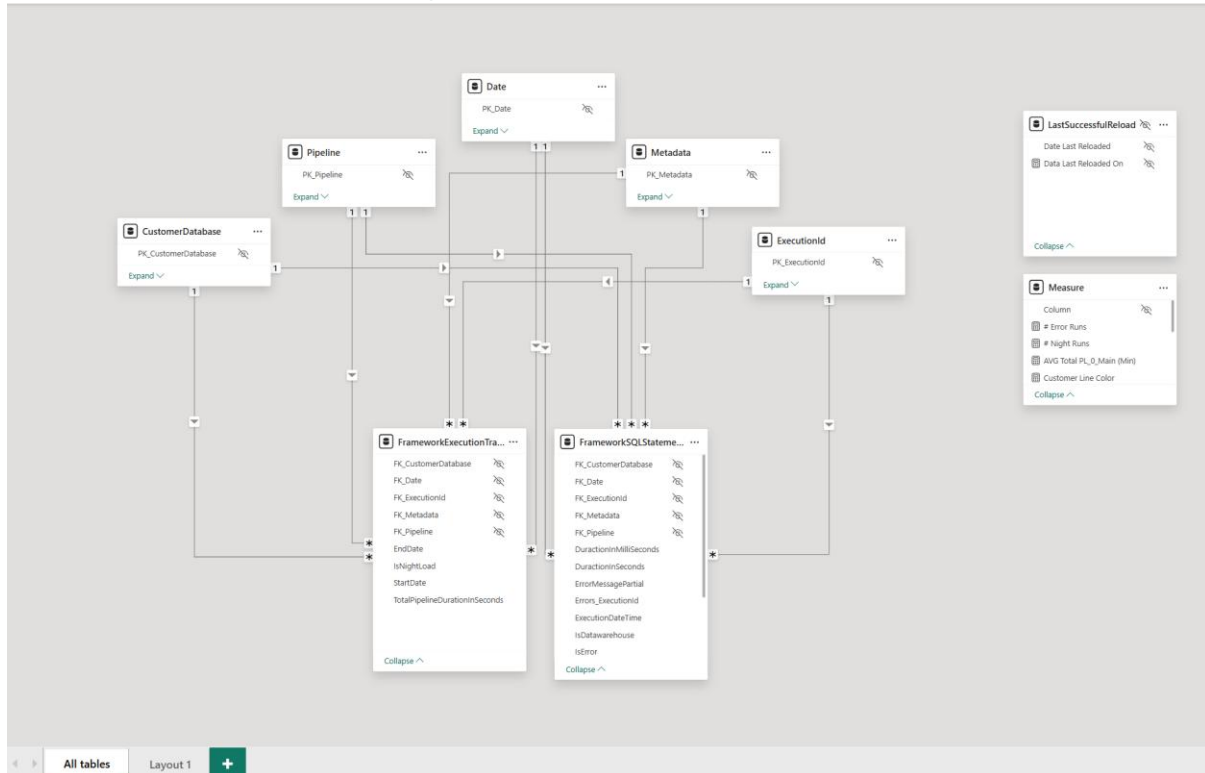
This Power BI model connects to these two log tables (as Facts) and to several dimension tables derived from the framework metadata.

2.3 Why PL_3 Data is One Day Behind

Important: this report itself is part of PL_3_Serve. That means when the report refreshes tonight, PL_3 of *tonight* has not finished yet (it is still running). The most recent PL_3 duration data will always be from the *previous night*. PL_1 and PL_2 data will be from the current night, as they complete before PL_3 starts. That's why you will see that current logging is done on "PL_1 + PL_2" instead of just PL_0_Main.

3. Data Model

The model follows a star schema pattern with 2 fact tables and 5 dimension tables.



Below is a summary:

3.1 Fact Tables

Table	Description
FrameworkExecutionTrace	One row per pipeline execution per customer. Contains TotalPipelineDurationInSeconds, StartDate, EndDate, <i>IsNightLoad</i> .
FrameworkSQLStatementTrace	One row per SQL statement trace. Contains DurationInMilliseconds, PrintStatementPartial, ErrorMessagePartial, IsError, <i>IsNightLoad</i> , plus flags for pipeline stage (IsIngestion, IsDatawarehouse, IsServe).

3.2 Dimension Tables

Table	Key Attributes
Pipeline	PK_Pipeline, BK_Pipeline, Pipeline (name: PL_1_Ingestion, PL_2_Datawarehouse, PL_3_Serve & child pipelines)
Metadata	PK_Metadata, BK_Metadata, DatabaseServerName, SourceApplicationName, TableType, SourceDetailName, DestinationDatabaseTableName, TableSizeMB, RowsInTable, CompressionType, SlowlyChangingDimensionType, IsIncrementalLoad, FlagResetTable, IsActive

ExecutionId	PK_ExecutionId, BK_ExecutionId, ExecutionId, DatabaseServerName, IsNightLoad , IsError, StartDate, EndDate, IsLastSuccessfulNightLoad
Date	Full calendar dimension with EN/FR labels, week/month/quarter/year hierarchies, and utility flags (Is YTD, Is Last 2 Years, etc.)
CustomerDatabase	CustomerDatabase, CustomerReportName, PrimaryColorHex, SecondaryColorHex, LogoURL. <i>This is the main slicer for selecting a client.</i>

3.3 Relationships

Both fact tables have 5 foreign keys each (FK_CustomerDatabase, FK_Pipeline, FK_Metadata, FK_Date, FK_ExecutionId), connecting to the corresponding PK in each dimension. All relationships are active, many-to-one.

4. Assignment Deliverables

Below are the expected deliverables. Each deliverable is a report page (or a set of measures) in the *.pbix* file. Read each section carefully and ask questions if anything is unclear.

4.1 Measures: Clean Foundation

Priority: before building any visuals, you always need to define a clean set of base measures.

The current measures in the model are quick first tests. They can be deleted or reworked. Your job is to create a proper, well-structured measure set that will serve as the foundation for all report pages.

Critical: Filter to Night Loads Only

The ExecutionId dimension has a column **IsNightLoad**. This is crucial. Without filtering on **IsNightLoad = TRUE**, your measures will include all logging: testing pipelines, development work, ad-hoc runs, etc. This distorts every KPI.

Incorporate this filter into your base measures so all downstream measures inherit it. Think about how to structure this efficiently (hint: a base measure with **CALCULATE + IsNightLoad** filter, then build time intelligence and variants on top).

Interesting Measures (or that we think could be interesting)

- Total Pipeline Duration (minutes) filtered to night loads
- Duration per pipeline stage: PL_1, PL_2, PL_3 separately
- Combined PL_1 + PL_2 duration
- Number of night runs (distinct ExecutionId count)
- Average pipeline duration per night run
- Number of error runs
- Total step duration (seconds, from SQLStatementTrace)
- Last 7 days average per load (for use in the long-running loads page)

All measures must live in a dedicated **Measure** table (already present in the model). Use display folders to organise them (e.g. 'Pipeline Duration', 'Errors', 'Step Duration').

Refer to the DAX Coding Standards appendix at the end of this document for naming, formatting, and variable conventions.

Only make measures that will fit the questions following 4.2 and onwards.

4.2 Source Applications Overview

We want a report page that shows which `SourceApplicationName` values are being used across our clients. This comes from the Metadata dimension.

Expect NULLs here. Not every metadata entry has a `SourceApplicationName` filled in. This is acceptable, but we want visibility into it.

Data Quality sub-page

Create a second page (or a section on the same page) acting as a **Data Quality** sheet:

- Show which clients have missing (NULL) `SourceApplicationName` values
- Show how many metadata entries per client have missing application names
- This helps us prioritise cleanup: if a client has 50 tables without an application name, that is worth fixing

Think about the visual

A matrix with *CustomerDatabase* on rows and *SourceApplicationName* on columns (with count of metadata entries) is a good starting point. For the data quality view, a simple table showing Client + Count of Missing + Total metadata rows is effective, but don't forget an ability to drill-down to the details.

4.3 Front Sheet: Per-Customer Pipeline Duration

This is the main landing page of the report. It should give an immediate overview of pipeline performance for all active customers.

Requirements

For each active customer, show the duration of **PL_1**, **PL_2**, and **PL_3**.

Remember the timing nuance from Section 2.3:

- **PL_1 and PL_2:** data from last night (the most recent completed run)
- **PL_3:** data from the night before (because this report is part of PL_3 and has not finished yet)

You will need to create specific measures that handle this offset for PL_3.

Visual Design Hint

We want a combined visual per customer. Think of a small card-like layout for each customer, inspired by KPI cards with a sparkline (see the example screenshot attached to this assignment). The idea:

- A KPI value at the top showing the total duration (e.g. PL_1 + PL_2 combined)
- A small line chart (sparkline style) below showing the last 7 days trend
- This gives both a current snapshot and a trend feel in one compact visual

Research how to achieve this. Suggested resource:

YouTube: "How to Power BI" channel, specifically this playlist:

[How to Power BI - Visual Design Playlist](#)

Visual approach

Consider using a combination of Card visuals and Line charts inside a container, or explore the built-in KPI visual. The small multiples feature might also work well here. There is no single 'right answer', find something that is clean and informative.



Example KPI & Chart visual.

4.4 Long-Running Loads Analysis

This page should help identify individual loads (at the MetadataId level) that are slow and potentially easy to improve.

Requirements

- Show the last 7 days average duration per metadata entry (table/stored procedure)
- Include the DestinationDatabaseTableName from the Metadata dimension so users can identify the specific table
- Sort by duration descending (slowest first)

Easy Improvement Indicators

Alongside the duration data, include columns from the Metadata dimension that hint at easy improvements:

Indicator	What It Tells Us
TableSizeMB (small)	If a table is small in size but takes long to load, the stored procedure logic is likely complex. This is harder to fix but worth flagging.
RowsInTable (low)	Same logic: few rows but slow load indicates complex transformations.
FlagResetTable = 1	This means the table is fully truncated and reloaded every night instead of using incremental load. Switching to incremental (FlagResetTable = 0 with proper UniqueColumnNames) is often an easy performance win.

Consultant value

This page is not just for monitoring. It is a tool for us to propose improvements to clients. If we see a table with FlagResetTable = 1, a small table size, and a long duration, we can suggest an incremental load approach or a stored procedure review.

4.5 Metadata Tables & Settings

Create a page that makes it easy to select a specific customer and then immediately see:

- What tables exist for that customer (DestinationDatabaseTableName)
- Which are the biggest tables (TableSizeMB, RowsInTable)
- What their source is (SourceApplicationName, SourceDetailName, TableOrigin)
- Compression type, number of indexes, SCD type
- Whether incremental load is enabled (IsIncrementalLoad)
- Whether the table is active (IsActive)

This is a reference page. It should be dense with data but well-organised. A matrix or detailed table visual with a prominent CustomerDatabase slicer is the expected approach.

Design tip

Use conditional formatting on TableSizeMB (color scale) to make large tables visually stand out. Group or sort by SourceApplicationName so users can quickly scan what sources feed into the platform.

5. Deliverables Summary

#	Deliverable	Key Focus	Status
4.1	Clean Measures	IsNightLoad filter, DAX standards, display folders	
4.2	Source Applications	Overview + Data Quality (NULL analysis)	
4.3	Front Sheet	Per-customer KPI + sparkline, PL_3 offset	
4.4	Long-Running Loads	7-day average, improvement indicators	
4.5	Metadata Explorer	Customer-filtered table details	

5.1 Approach

Work through the deliverables in order: measures first (4.1), then build the pages (4.2 through 4.5). Start simple, iterate, and ask questions when needed.

When presenting your work, be prepared to explain your design choices. Why did you pick that visual? Why did you structure the measures that way? This is part of being a consultant.

6. Resources

6.1 Power BI Visual Design

- **How to Power BI (YouTube):** excellent channel for report design inspiration and techniques

[Visual Design Playlist](#)

6.2 DAX Reference

- **SQLBI (sqlbi.com):** Marco Russo & Alberto Ferrari, the definitive DAX resource
- **DAX.do:** online DAX sandbox for testing measure logic
- **Data Simplified DAX Coding Standards:** see Appendix A of this document

6.3 DS Framework Documentation

For framework architecture questions, refer to the internal DS Framework documentation or ask Bavo directly. The key concepts you need for this assignment (pipeline stages, metadata structure, logging) are covered in Section 2 of this document.

Appendix A: DAX Coding Standards

Source: ds-powerbi skill, created by **Pieter Pype** (Data Simplified).

A.1 Formatting & Indentation

- One clause per line (CALCULATE, FILTER, ALL, etc.)
- Indent nested functions by 4 spaces
- Opening parenthesis stays on the same line as the function name
- Closing parenthesis on its own line, aligned with the function
- The measure name must always be alone on line 1; the expression starts on line 2

Example:

```
Sales YTD =  
CALCULATE(  
    [Total Sales],  
    DATESYTD( 'Date'[Date] )  
)
```

A.2 Naming Conventions

Pattern: [Subject] [Object] [Aggregation/Type]

Type	Convention	Example
Base measure	Title Case, suffix = aggregation	Pipeline Duration Sum
Variance	Suffix with Var or Var %	Pipeline Duration Var %
Prior period	Suffix with PY / PM	Pipeline Duration Sum PY
Time intelligence	Suffix with YTD, Rolling 12M, etc.	Error Count YTD
Hidden / helper	Prefix with _	_NightLoadFilter

A.3 Variable Usage

- **Use VAR / RETURN** when the same expression is used more than once, or when logic has more than two steps
- **Variable names:** prefix with _, use CamelCase: _TotalSales, _FilteredRows, _Delta

A.4 Division

Always use DIVIDE(). Never use the / operator. DIVIDE handles division by zero gracefully (returns BLANK).

A.5 Comments

- Always include a one-line comment above the measure explaining its purpose
- Use inline comments for non-obvious logic

Example:

```
-- Total pipeline duration in minutes, filtered to night loads only
Pipeline Duration Night Load Sum =
CALCULATE(
    SUM( FrameworkExecutionTrace[TotalPipelineDurationInSeconds] ) / 60,
    ExecutionId[IsNightLoad] = TRUE()
)
```

A.6 Measure Organisation

- **All measures live in a dedicated Measure table** (already present in this model)
- Use display folders to group measures (e.g. Pipeline Duration, Errors, Step Duration)
- Always set the appropriate number format in the measure properties

A.7 Visual Design Rules

Rule	Detail
Never use pie/donut charts	Use stacked bar charts for part-to-whole comparisons
Disable gridlines on line charts	Format pane > Gridlines > toggle off (both horizontal and vertical)
Slicers on the left	Never on top, never floating
Top-left = most important	Users scan top-left first; place key KPIs there
One story per page	Do not overload a single page with unrelated visuals
Always label axes	Never leave users guessing what a number represents

A.8 Conditional Formatting Colors

Meaning	Hex	When to Use
Positive / good	#24A625	Performance improvements, successful loads
Negative / bad	#FF0000	Errors, degraded performance
Neutral	#808080	No change, informational

Contact

Bavo Vandekeere *

Data Simplified (BV)

bavo.vandekeere@data-simplified.com | +32493521618

Grote Steenweg 36 | 0101, 9870 Olsene

() Vaste vertegenwoordiger van BIDA Consulting BV, bestuurder*